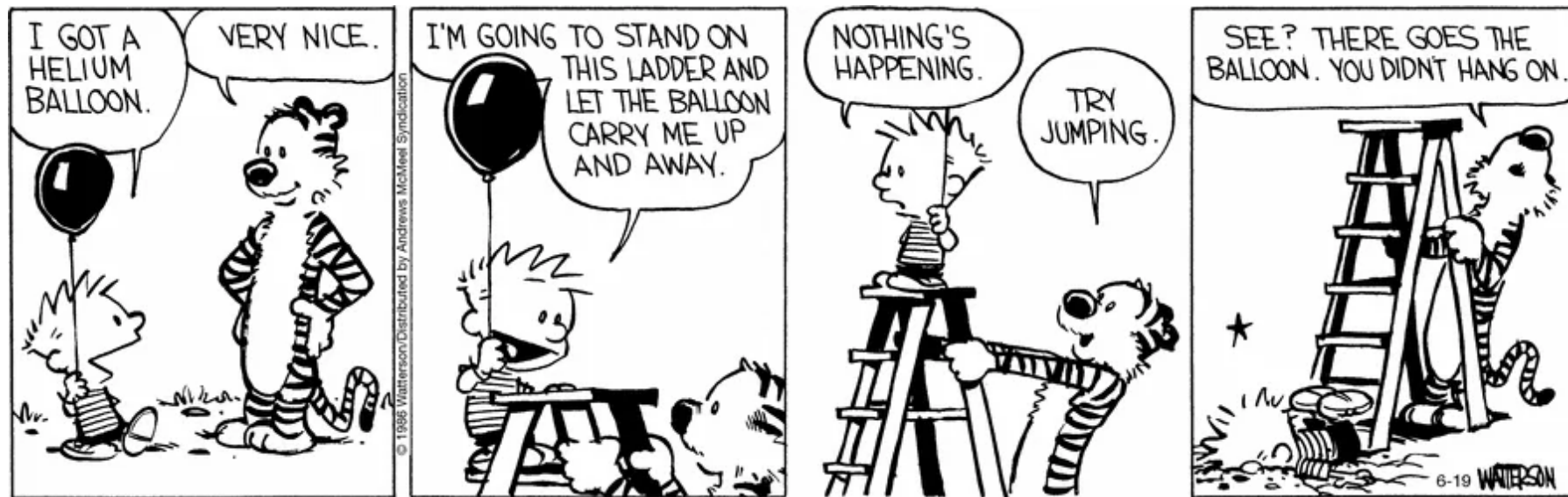


Lecture 10: Dictionaries



INFO 1100: Introduction to Programming

Let's look at more ways of
combining and ordering data!

Dictionaries

Dictionaries

What do they do?

Dictionaries

*maybe my favorite data
structure...*

Dictionaries

a **mapping** of one set of
data to another

Dictionaries

lists map indices to elements

[	'a'	,	'b'	,	'c'	,	'd'	,	'e'	]
	0		1		2		3		4	

Dictionaries

what if we want to map from
something other than indices?

Dictionaries

what if we want to map from
something other than indices?

*words to
definitions*

*state codes to
state names*

*names to
phone numbers*

Dictionaries

a dictionary maps
keys to **values**

Dictionaries

```
{ "AL": "Alabama", "AK": "Alaska",  
  "AZ": "Arizona", "AR": "Arkansas" }
```

Dictionaries

keys

```
{ "AL": "Alabama", "AK": "Alaska",  
  "AZ": "Arizona", "AR": "Arkansas" }
```

Dictionaries

keys

```
{ "AL": "Alabama", "AK": "Alaska",  
  "AZ": "Arizona", "AR": "Arkansas" }
```

keys can be any immutable type

Dictionaries

values

```
{ "AL": "Alabama", "AK": "Alaska",  
  "AZ": "Arizona", "AR": "Arkansas" }
```

Dictionaries

values

```
{ "AL": "Alabama", "AK": "Alaska",  
  "AZ": "Arizona", "AR": "Arkansas" }
```

values can be any type

Dictionaries

an **item**

```
{ "AL": "Alabama", "AK": "Alaska",  
  "AZ": "Arizona", "AR": "Arkansas" }
```

Dictionaries

keys must be unique

Dictionaries

keys must be unique

*otherwise, how will we know what
values they are associated with?*

Dictionaries

Enter in the chat:

**What data would you want to
store in a dictionary?**

Working with dictionaries

```
states = {}
```

Create an empty
dictionary using empty
brackets

Working with dictionaries

```
states = {}  
  
states["TX"] = "Texas"  
  
states["MN"] = "Minnesota"
```

Subscript assignment
creates a new dictionary
item if the key doesn't
already exist

Working with dictionaries

```
states = {}  
  
states["TX"] = "Texas"  
states["MN"] = "Minnesota"  
print(states)
```

Working with dictionaries

```
states = {}
```

```
states["TX"] = "Texas"
```

```
states["MN"] = "Minnesota"
```

```
print(states) → {"TX": "Texas", "MN": "Minnesota"}
```

Working with dictionaries

```
states = {}  
  
states["TX"] = "Texas"  
states["MN"] = "Minnesota"  
print(states)  
print(states["TX"])
```

Subscripting **looks up** the
the value for a given key

Working with dictionaries

```
states = {}  
  
states["TX"] = "Texas"  
states["MN"] = "Minnesota"  
print(states)  
print(states["TX"]) → "Texas"
```

Working with dictionaries

```
states = {}  
  
states["TX"] = "Texas"  
states["MN"] = "Minnesota"  
print(states)  
print(states["TX"])  
print(states["IL"])
```

Subscript lookup causes a **KeyError** if the key is not in the dictionary

Working with dictionaries

```
states = {}  
  
states["TX"] = "Texas"  
states["MN"] = "Minnesota"  
print(states)  
print(states["TX"])  
print(states["IL"]) → KeyError: 'IL'
```

Working with dictionaries

```
states = {}  
  
states["TX"] = "Texas"  
states["MN"] = "Minnesota"  
print(states)  
print(states["TX"])  
print(len(states))
```

The **len()** function
counts the # of items
in a dictionary

Working with dictionaries

```
states = {}  
  
states["TX"] = "Texas"  
states["MN"] = "Minnesota"  
print(states)  
print(states["TX"])  
print(len(states)) → 2
```

The **len()** function
counts the # of items
in a dictionary

Working with dictionaries

```
states = {}  
  
states["TX"] = "Texas"  
states["MN"] = "Minnesota"  
print(states)  
print(states["TX"])  
print(len(states))  
print("TX" in states)
```

in returns membership
in dictionary keys

Working with dictionaries

```
states = {}  
  
states["TX"] = "Texas"  
states["MN"] = "Minnesota"  
print(states)  
print(states["TX"])  
print(len(states))  
print("TX" in states) → True
```

in returns membership
in dictionary keys

Working with dictionaries

```
states = {}  
  
states["TX"] = "Texas"  
states["MN"] = "Minnesota"  
print(states)  
print(states["TX"])  
print(len(states))  
print("IL" in states) → False
```

in returns membership
in dictionary keys

Working with dictionaries

```
states = {}  
  
states["TX"] = "Texas"  
states["MN"] = "Minnesota"  
print(states)  
print(states["TX"])  
print(len(states))  
print("Texas" in states) → False
```

in returns membership
in dictionary *keys*

Working with dictionaries

```
states = {"IL": "Illinois",  
          "NY": "New York"}
```

You can create
dictionaries that
contain items

Working with dictionaries

```
states = {"IL": "Illinois",  
          "NY": "New York"}  
  
states["IL"] = "Ill"
```

You can also **change** a value with subscripting assignment

Working with dictionaries

```
states = {"IL": "Illinois",  
          "NY": "New York"}
```

```
states["IL"] = "Ill"
```

```
print(states) → {"IL": "Ill", "NY": "New York"}
```

Working with dictionaries

```
states = {"IL": "Illinois",  
          "NY": "New York"}  
  
states["IL"] = "Ill"  
print(states)  
print(states.pop("IL"))
```

The `.pop()` function takes a key as argument, removes a key-value pair, and returns the value

Working with dictionaries

```
states = {"IL": "Illinois",  
          "NY": "New York"}  
  
states["IL"] = "Ill"  
  
print(states)  
  
print(states.pop("IL")) → "Ill"
```

Working with dictionaries

```
states = {"IL": "Illinois",  
          "NY": "New York"}  
  
states["IL"] = "Ill"  
  
print(states)  
  
print(states.pop("IL"))  
  
print(states) → {"NY": "New York"}
```

Working with dictionaries

```
starter = dict()
```

We can also use `dict()`
to create an empty
dictionary

Working with dictionaries

```
starter = dict()  
print(starter)
```

Working with dictionaries

```
starter = dict()
```

```
print(starter) → {}
```

Working with dictionaries

```
data = {'a': 1, 'b': 2}
data_again = dict(data)
```

dict() can also copy
dictionaries

Working with dictionaries

```
data = { 'a': 1, 'b': 2 }
```

```
data_again = dict(data)
```

```
data[ 'c' ] = 3
```

Working with dictionaries

```
data = { 'a': 1, 'b': 2 }  
data_again = dict(data)  
data[ 'c' ] = 3  
print(data)
```

Working with dictionaries

```
data = { 'a': 1, 'b': 2 }
```

```
data_again = dict(data)
```

```
data['c'] = 3
```

```
print(data) → {"a": 1, "b": 2, "c": 3}
```

Working with dictionaries

```
data = { 'a': 1, 'b': 2 }  
data_again = dict(data)  
data['c'] = 3  
print(data)  
print(data_again)
```

Working with dictionaries

```
data = { 'a': 1, 'b': 2 }
```

```
data_again = dict(data)
```

```
data[ 'c' ] = 3
```

```
print(data)
```

```
print(data_again) → {"a": 1, "b": 2}
```

Working with dictionaries

```
data = { 'a': 1, 'b': 2 }  
data_again = dict(data)  
data['c'] = 3  
print(data)  
print(data_again)
```

We can also use
.values() and **.keys()**
to access only the values
or keys

Working with dictionaries

```
data = { 'a': 1, 'b': 2 }  
data_again = dict(data)  
data[ 'c' ] = 3  
print(data)  
print(data_again)  
print(data.values())
```

Working with dictionaries

```
data = { 'a': 1, 'b': 2 }
```

```
data_again = dict(data)
```

```
data[ 'c' ] = 3
```

```
print(data)
```

```
print(data_again)
```

```
print(data.values()) → dict_values([1, 2, 3])
```

Working with dictionaries

```
data = { 'a': 1, 'b': 2 }  
data_again = dict(data)  
data[ 'c' ] = 3  
print(data)  
print(data_again)  
print(data.values())  
print(data.keys())
```

Working with dictionaries

```
data = { 'a': 1, 'b': 2 }  
data_again = dict(data)  
data['c'] = 3  
print(data)  
print(data_again)  
print(data.values())  
print(data.keys()) → dict_keys(['a', 'b', 'c'])
```

```
d = {1: "one", 2: "two"}  
d["three"] = 3  
print(d[3])
```

What prints?

- a) 3
- b) 'three'
- c) None
- d) **KeyError**
- e) **ValueError**

Working with dictionaries

```
contronyms = {  
    'nonplussed':  
        ['surprised and confused',  
         'unperturbed'],  
    'sanction':  
        ['penalty',  
         'official permission']  
}
```

Sometimes we
want to use
lists as values

Working with dictionaries

```
students = {  
    'ewe2': {  
        'age': 19,  
        'courses':  
            ['CS 1110', 'MATH 1920']  
    },  
    'top20': {  
        'age': 21  
        'courses':  
            ['INFO 1100', 'HADM 4300']  
    }  
}
```

And often we
want to nest
dictionaries

Working with dictionaries

We can, and often do, use dictionaries as accumulators

Working with dictionaries

```
def char_counts(s):  
    pass
```

We can, and often do, use dictionaries as accumulators

Working with dictionaries

```
def char_counts(s):  
    counts = {}
```

We can, and often do, use dictionaries as accumulators

Working with dictionaries

```
def char_counts(s):  
    counts = {}  
    for char in s:  
        pass
```

We can, and often do, use dictionaries as accumulators

Working with dictionaries

```
def char_counts(s):  
    counts = {}  
    for char in s:  
        if char in counts:  
            pass  
        else:  
            pass
```

We can, and often do, use dictionaries as accumulators

Working with dictionaries

```
def char_counts(s):  
    counts = {}  
    for char in s:  
        if char in counts:  
            counts[char] += 1  
        else:  
            pass
```

We can, and often do, use dictionaries as accumulators

Working with dictionaries

```
def char_counts(s):  
    counts = {}  
    for char in s:  
        if char in counts:  
            counts[char] += 1  
        else:  
            counts[char] = 1
```

We can, and often do, use dictionaries as accumulators

Working with dictionaries

```
def char_counts(s):  
    counts = {}  
    for char in s:  
        if char in counts:  
            counts[char] += 1  
        else:  
            counts[char] = 1  
    return counts
```

We can, and often do, use dictionaries as accumulators

Working with dictionaries

```
def print_counts(counts):  
    for char in counts:  
        count = counts[char]  
        print(char + ' ' +  
              '*' * count)
```

We can also use for loops to iterate over dictionary keys

Working with dictionaries

Enter in the chat:

**What dictionary functionality
do you think is most useful?**

Example 1

```
def get_price(menu, item):  
    if item in menu:  
        return menu[item]  
    return 0.0  
  
prices = {"coffee": 3.5, "tea": 2.0, "muffin": 4.0}  
print(get_price(prices, "tea"))  
print(get_price(prices, "juice"))
```

Example 2

```
def most_stocked(inventory):  
    best_item = None  
    best_count = 0  
    for item in inventory:  
        if inventory[item] > best_count:  
            best_item = item  
            best_count = inventory[item]  
    return best_item  
  
warehouse = {"apples": 50, "pears": 20, "plums": 30}  
print(most_stocked(warehouse))
```

Example 3

```
def invert(mapping):  
    result = {}  
    for key in mapping:  
        value = mapping[key]  
        result[value] = key  
    return result  
  
print(invert({"a": 1, "b": 2, "c": 3}))
```

Example 4

```
def group_by_length(words):  
    groups = {}  
    for word in words:  
        length = len(word)  
        if length in groups:  
            groups[length].append(word)  
        else:  
            groups[length] = [word]  
    return groups  
  
print(group_by_length(["hi", "cat", "dog", "a", "yes"]))
```

Example 5

```
def city_pop(data, country, city):
    if country not in data:
        return None
    if city not in data[country]:
        return None
    return data[country][city]

world = {"France": {"Paris": 2100000, "Lyon": 500000},
        "Japan": {"Tokyo": 14000000}}

print(city_population(world, "France", "Lyon"))
print(city_population(world, "France", "Nice"))
print(city_population(world, "Brazil", "Rio"))
```

Example 6

```
def deep_sum(data):
    total = 0
    for key in data:
        if isinstance(data[key], dict):
            total += deep_sum(data[key])
        else:
            total += value
    return total

tree = {"a": 1, "b": {"c": 2, "d": {"e": 3,
    "f": 4}}, "g": 6}
print(deep_sum(tree))
```

Questions?

(... I'll wait for 2)

For tomorrow:

(07/10 @ 11:30am)

- Lab 10
- Quiz tomorrow!

For tomorrow:
(07/10 @ 11:30am)

- Lab 10
- Quiz tomorrow!

Monday:
(07/13 @ 9:00am)

Assignment 3

Tuesday:
(07/14 @ 11:30am)

Readings + GRQs